

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: XỬ LÝ ẢNH SỐ

PHÂN ĐOẠN TỪNG ĐỐI TƯỢNG BẰNG DEEP LEARNING VỚI STARDIST

Đối chiếu Fiji/ImageJ và cài đặt Python

Mã nhóm: **GR03**

Thành viên: Nguyễn Hồng Quang – B23DCVT361

Nguyễn Trọng Nam Khánh – B23DCCE052

Vũ Dũng – B23DCVT110

HÀ NỘI

Tóm tắt

Báo cáo đánh giá StarDist 2D pretrained cho phân đoạn nhân tế bào, so sánh với Otsu và marker-controlled Watershed trên 50 ảnh test DSB2018. StarDist đạt AP@0.50 0.846 ± 0.132 và PQ@0.50 0.782 ± 0.117, cao hơn hai baseline. Đối chiếu trực tiếp với Fiji/ImageJ trên ba ca cố định cho thấy hai ca đạt tiêu chí F1 và Dice ≥ 0.95; ca low không đạt. Pipeline, nhãn TIFF, cấu hình và kết quả được lưu trong repository. Thí nghiệm không huấn luyện lại CNN và chưa xác minh test split độc lập với dữ liệu huấn luyện của checkpoint.

Từ khóa: StarDist, instance segmentation, U-Net, star-convex, Fiji, ImageJ, Otsu, Watershed, Panoptic Quality.

Mục lục

Tóm tắt i

Danh mục hình vẽ iv

Danh mục bảng v

1 Giới thiệu 1

1.1 Bài toán và mục tiêu . . . 1

2 Cơ sở lý thuyết 2

2.1 Bài toán instance segmentation . . . 2

2.1.1 Tiền xử lý và quy ước số học . . . 2

2.2 CNN và U-Net . . . 3

2.2.1 Tích chập, receptive field và phi tuyến . . . 3

2.2.2 U-Net làm backbone đa đầu ra . . . 3

2.3 Probability map và star-convex polygon . . . 3

2.3.1 Giả thiết star-convex và target radial distance . . . 3

2.3.2 Khả năng biểu diễn và giới hạn . . . 4

2.4 Huấn luyện và NMS . . . 4

2.4.1 Hàm mất mát đa nhiệm . . . 4

2.4.2 Suy luận, candidate filtering và polygon NMS . . . 4

2.5 Baseline Otsu và Watershed . . . 5

2.5.1 Otsu và connected components . . . 5

2.5.2 Marker-controlled Watershed . . . 5

3 Fiji/ImageJ làm nền tảng tham chiếu 6

3.1 Plugin và cấu hình . . . 6

3.1.1 Thông số có ý nghĩa thực nghiệm . . . 6

3.2 Thực thi và tái lập . . . 7

3.2.1 Quy trình GUI . . . 7

3.2.2 Đo định lượng trong Fiji và giới hạn hiện tại . . . 7

3.3 Kiểm chứng Fiji–Python . . . 8

4 Cài đặt Python **9**
4.1 Kiến trúc phần mềm . . . 9
4.2 Phần nhóm tự cài đặt và phần tái sử dụng . . . 9
4.3 Cài đặt các bước hình học . . . 10
4.3.1 Bản đồ khoảng cách chuẩn hóa . . . 10
4.3.2 Tia, đa giác và NMS tham chiếu . . . 10
4.3.3 Baseline và metric do nhóm lập trình . . . 10
4.3.4 Ví dụ kiểm tra thủ công . . . 11
4.4 Môi trường . . . 11

5 Thiết kế thí nghiệm và metric **12**
5.1 Dữ liệu và cấu hình . . . 12
5.2 Matching và metric . . . 12
5.3 Giao thức đánh giá . . . 13

6 Kết quả và thảo luận **14**
6.1 So sánh trên 50 ảnh test . . . 14
6.2 Phân tích kết quả . . . 17
6.3 Đối chiếu trực tiếp Fiji–Python . . . 17
6.4 Ảnh hưởng ngưỡng xác suất và NMS . . . 19

7 Kết luận **21**
7.1 Giới hạn . . . 21
7.2 Hướng phát triển . . . 21

A Phân công công việc **23**

B Tái lập thực nghiệm **24**
B.1 Cài môi trường và kiểm thử . . . 24
B.2 Tạo lại kết quả và artefact . . . 24
B.3 Mã nguồn tham chiếu . . . 24

Danh sách hình vẽ

3.1 Minh chứng từng bước của workflow Fiji/ImageJ trên ca median. Từ trái sang phải: input TIFF, ảnh hiển thị percentile 1/99.8 (minh hoạ), Label Image TIFF StarDist, overlay prediction/ground truth và tham số/runtime từ log. 8

6.1 Mean AP@0.50 và PQ@0.50 trên 50 ảnh test DSB2018. Độ lệch chuẩn được trình bày trong Bảng 6.1. 15
6.2 Ca median được giữ cố định từ lần chạy gốc có clipping, không chọn lại theo kết quả mới. Màu biểu diễn ID instance, không phải lớp semantic. Otsu gộp nhiều nhân tiếp xúc; Watershed tách được một phần; StarDist bám theo instance ground truth sát hơn. 15
6.3 Ca low cố định từ lần chạy gốc; tên ca không biểu thị thứ hạng AP sau cập nhật. 15
6.4 Ba output Label Image thực tế của plugin StarDist Fiji/ImageJ: ca low, median và high đã chọn trước, đặt cạnh input và ground truth. Mỗi màu là một ID instance. 16
6.5 Sai khác trực tiếp giữa hai đầu ra. ID nhãn được ghép một-một trước khi đo pixel agreement trên hợp foreground, tránh kết luận sai do đổi thứ tự ID. 18
6.6 Probability sweep trên 30 ảnh validation; error bar là SD giữa ảnh. 19
6.7 NMS sweep trên cùng validation, giữ probability mặc định. 19
6.8 Ví dụ ở ID validation đầu tiên đã chốt, không chọn theo score. Hai đầu mút probability/NMS kèm FP/FN minh hoạ candidate được giữ/bỏ khi đổi ngưỡng. Màu là ID instance; kết luận định lượng vẫn dựa toàn bộ 30 ảnh. 20

Danh sách bảng

6.1 Metric instance trên cùng 50 test images. Fiji được đánh giá riêng ở bảng đối chiếu. 14
6.2 Metric foreground, sai số đếm và runtime Python/baseline, mean ± SD trên 50 ảnh. 14

Chương 1

Giới thiệu

1.1 Bài toán và mục tiêu

Ảnh hiển vi thường chứa nhiều nhân tế bào sát nhau. Mask nhị phân có thể gộp các nhân tiếp xúc, làm sai số đếm và phép đo hình thái. StarDist biểu diễn mỗi nhân bằng đa giác star-convex và dùng Non-Maximum Suppression (NMS) để loại dự đoán trùng [6].

Nhóm xây dựng pipeline Python gồm baseline, các hàm hình học tham chiếu và bộ đánh giá instance; dùng plugin Fiji/ImageJ để đối chiếu triển khai. Báo cáo tập trung vào ba câu hỏi:

RQ1. StarDist có tách nhân tiếp xúc tốt hơn Otsu và Watershed trên 50 ảnh test không?

RQ2. Nhãn Fiji và Python tương ứng đến mức nào trên ba ca cố định?

RQ3. Ngưỡng xác suất và NMS ảnh hưởng kết quả ra sao trên 30 ảnh validation?

Chương 2

Cơ sở lý thuyết

2.1 Bài toán instance segmentation

Cho ảnh $I : \Omega \to \mathbb{R}^C$, với miền pixel $\Omega = \{0, \dots, H-1\} \times \{0, \dots, W-1\}$ và C kênh. Ground truth là ảnh nhãn nguyên $Y : \Omega \to \{0, 1, \dots, M\}$: $Y(q) = 0$ là nền; mọi pixel mang cùng nhãn $m > 0$ tạo thành instance G_m. Mục tiêu là ước lượng $\hat{Y} = f_\theta(I)$ sao cho từng object có ID riêng, kể cả khi hai object cùng lớp chạm biên.

- **Semantic segmentation** dự đoán foreground/background hoặc lớp tại mỗi pixel; hai nhân chạm nhau vẫn có thể là một vùng liên thông.
- **Object detection** trả bounding box và score, hữu ích để định vị nhưng không đủ cho diện tích, chu vi hay hình dạng nhân.
- **Instance segmentation** trả tập mask/polygon $\{P_j\}_{j=1}^{N}$; các P_j được phân biệt theo ID và là đầu vào đúng cho phép đếm.

Dice foreground không phân biệt các ID, nên cần đánh giá thêm bằng ghép cặp instance theo IoU để nhận diện lỗi tách và gộp.

2.1.1 Tiền xử lý và quy ước số học

Các model pretrained nhạy với thang cường độ. Với percentile thấp/cao a, b, chuẩn hoá dùng trong thí nghiệm là

$$I_{\text{norm}}(q) = \frac{I(q) - Q_a(I)}{Q_b(I) - Q_a(I) + \varepsilon}, \tag{2.1}$$

trong đó Q_a, Q_b là percentile và ε tránh chia cho 0. Hai nền tảng cùng dùng $a = 1$, $b = 99.8$, nhưng Python dùng percentile tuyến tính NumPy và không clipping; CSBDeep Java dùng histogram percentile, chặn dưới ở 0 và không chặn trên. Đầu ra là TIFF nhãn nguyên 2D; màu chỉ dùng để hiển thị ID.

2.2 CNN và U-Net

2.2.1 Tích chập, receptive field và phi tuyến

CNN học các kernel thay vì thiết kế thủ công feature. Với feature map đầu vào $a^{(l-1)}$ và kernel $w^{(l)}$, giá trị ở lớp l là

$$a^{(l)}_{i,j,c} = \phi\left(b^{(l)}_c + \sum_{u,v,c'} w^{(l)}_{u,v,c',c} a^{(l-1)}_{i+u,j+v,c'}\right). \tag{2.2}$$

Hàm phi tuyến ϕ (ví dụ ReLU) cho phép xấp xỉ quan hệ phi tuyến giữa cấu trúc ảnh và nhãn. Khi nhiều lớp và pooling/strided convolution nối tiếp, một pixel đầu ra quan sát được vùng lân cận lớn hơn, gọi là *receptive field*. Ngữ cảnh này hữu ích để phân biệt đốm nhiễu với nhân tế bào, nhưng downsampling quá mạnh sẽ làm mất chi tiết biên.

2.2.2 U-Net làm backbone đa đầu ra

U-Net gồm encoder giảm dần độ phân giải để học ngữ cảnh, bottleneck, rồi decoder upsampling để khôi phục lưới pixel. Skip connection nối feature map cùng scale từ encoder sang decoder, giúp decoder dùng lại vị trí/biên mảnh thay vì chỉ dựa vào feature đã nén [5]. StarDist dùng backbone U-Net chia sẻ với hai head dự đoán trên lưới đầu ra:

$$(\hat{p}(q), \hat{\mathbf{r}}(q)) = h_\theta(I)(q), \qquad \hat{p}(q) \in [0,1], \quad \hat{\mathbf{r}}(q) \in \mathbb{R}^K_+. \tag{2.3}$$

Head probability trả một kênh score; head hình học trả K khoảng cách tia. Vì hai task dùng cùng feature, score có thể đánh giá độ tin cậy proposal còn distance head mô tả biên object. Mạng không dự đoán trực tiếp ID nhãn cho mỗi pixel.

2.3 Probability map và star-convex polygon

2.3.1 Giả thiết star-convex và target radial distance

Một tập S là star-convex đối với điểm c nếu mọi đoạn thẳng từ c đến điểm $x \in S$ đều nằm trong S, tức $[c, x] \subseteq S$. Đối với một pixel foreground q và K hướng đều nhau $\theta_k = 2\pi k/K$, target radial là

$$r_k(q) = \sup\{t \geq 0 : q + s(\cos\theta_k, \sin\theta_k) \in G(q)\ \forall s \in [0,t]\}, \quad k = 0, \ldots, K-1, \tag{2.4}$$

trong đó $G(q)$ là instance chứa q. Nói cách khác, ta đi từ q theo từng tia đến giao điểm đầu tiên với biên. Bài báo gốc dùng $K = 32$ [6]; K lớn hơn mô tả biên cong tốt hơn nhưng tăng số kênh, bộ nhớ và số polygon phải xử lý trong NMS.

Khi mạng dự đoán $\hat{\mathbf{r}}(q)$, các đỉnh polygon proposal là

$$v_k(q) = q + \hat{r}_k(q)(\cos\theta_k, \sin\theta_k). \tag{2.5}$$

Nối $v_0, \ldots, v_{K-1}$ theo thứ tự góc tạo polygon xấp xỉ instance. Một object có thể sinh proposal từ nhiều pixel bên trong nó, nhưng proposal gần tâm thường ổn định hơn. Target probability $p(q)$ là Euclidean distance transform (EDT) chuẩn hoá riêng trong từng instance: gần biên có giá trị xấp xỉ 0, trung tâm có giá trị cao. Do đó probability map vừa là foreground confidence vừa thiên về những tâm sinh polygon tốt.

2.3.2 Khả năng biểu diễn và giới hạn

Star-convex polygon phù hợp nhân tế bào/bào quan dạng khối tròn hoặc bầu dục, và tách hai nhân sát nhau bằng hai polygon chồng cạnh. Ngược lại, hình vòng, nhánh cây hoặc lõm sâu có thể không nhìn thấy toàn bộ biên từ một tâm; ray đi qua nền rồi vào lại object làm biểu diễn không đúng. Lỗi khi đó là hạn chế mô hình hình học, không chỉ là lỗi threshold.

2.4 Huấn luyện và NMS

2.4.1 Hàm mất mát đa nhiệm

Mạng học probability $\hat{p}$ và vector distance $\hat{\mathbf{r}}$. Với target $p, \mathbf{r}$, một dạng objective điển hình là

$$\mathcal{L}_{\text{prob}} = -\frac{1}{|\Omega|}\sum_{q\in\Omega}[p(q)\log\hat{p}(q) + (1-p(q))\log(1-\hat{p}(q))], \tag{2.6}$$

$$\mathcal{L}_{\text{dist}} = \frac{\sum_q p(q)\,\|\mathbf{r}(q) - \hat{\mathbf{r}}(q)\|_1}{\sum_q p(q) + \varepsilon}, \tag{2.7}$$

$$\mathcal{L} = \lambda_p\mathcal{L}_{\text{prob}} + \lambda_r\mathcal{L}_{\text{dist}}. \tag{2.8}$$

Trọng số $p(q)$ giảm tác động của pixel biên, nơi tia ngắn và nhạy với annotation, đồng thời nhấn vào các tâm. Thí nghiệm sử dụng trọng số pretrained để suy luận.

2.4.2 Suy luận, candidate filtering và polygon NMS

Tại inference, tạo candidate $C = \{q : \hat{p}(q) \geq t_p\}$. Mỗi $q \in C$ được decode theo (2.5); sau đó sắp giảm dần theo $\hat{p}(q)$. Với proposal được giữ A và proposal xét B, overlap là

$$\text{overlap}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}. \tag{2.9}$$

NMS giữ proposal score cao nhất còn lại và loại mọi proposal có overlap$(A, B) > t_{\text{nms}}$; lặp đến khi hết candidate. Độ đo này của StarDist 2D khác IoU dùng để đánh giá, vốn chuẩn hoá bằng hợp. Các polygon được giữ được raster hoá thành label ID không chồng nhau. Độ phức tạp CNN gần tuyến tính theo số pixel, còn hậu xử lý phụ thuộc số candidate và cặp polygon; vì vậy t_p, tile size và NMS ảnh hưởng runtime.

t_p thấp quá tạo nhiều positive giả và tăng chi phí NMS; cao quá làm mất object mờ. Ngưỡng t_{nms} thấp triệt proposal mạnh (dễ bỏ sót object sát nhau); cao dễ giữ trùng lặp. Các giá trị thực tế ở đây là $t_p = 0.479071$ và $t_{\text{nms}} = 0.3$, được lấy từ cấu hình model, không tune trên test.

2.5 Baseline Otsu và Watershed

2.5.1 Otsu và connected components

Otsu giả định histogram có thể tách thành hai lớp bằng ngưỡng t. Gọi ω_0, ω_1 là xác suất hai lớp và μ_0, μ_1 là trung bình cường độ của chúng, Otsu chọn ngưỡng toàn cục bằng cách cực đại hoá phương sai giữa lớp [4]:

$$t^* = \arg\max_t \ \sigma_B^2(t), \qquad \sigma_B^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2. \tag{2.10}$$

Sau threshold, lọc vùng nhỏ và connected components 8-neighbour tạo label. Điểm mạnh là không cần model, nhanh và có thể giải thích; điểm yếu là nền không đồng nhất/ảnh ít tương phản làm histogram không còn bimodal. Hai object chạm nhau thành một component nên Otsu có thể có Dice foreground tốt nhưng AP instance và count kém.

2.5.2 Marker-controlled Watershed

Watershed bổ sung bước tách object trên foreground mask F. Đầu tiên tính $D = \text{EDT}(F)$; đỉnh địa phương của D là marker gần tâm object. Thuật toán "ngập" landscape $-D$ từ các marker và đặt watershed line nơi hai vùng gặp nhau [8]. Pipeline triển khai là: (1) Otsu tạo F; (2) morphology/loại vùng nhỏ; (3) peak-local-max với khoảng cách tối thiểu d; (4) watershed của $-D$ bị ràng buộc trong F; (5) relabel output. Nó tách được nhân tiếp xúc tốt hơn Otsu nhưng over-segment khi marker nhiễu, under-segment khi hai đỉnh quá gần hoặc một nhân không có cực đại rõ. Vì vậy d phải chọn trên validation thay vì test.

Otsu và Watershed dựa trên cường độ và địa hình ảnh, còn StarDist sử dụng đặc trưng học từ dữ liệu cùng giả thiết star-convex. Cả ba phương pháp xuất nhãn TIFF và được đánh giá bằng cùng bộ metric.

Chương 3

Fiji/ImageJ làm nền tảng tham chiếu

3.1 Plugin và cấu hình

Fiji là bản phân phối ImageJ tích hợp các plugin khoa học. Plugin StarDist hỗ trợ model 2D pretrained/custom, chuẩn hóa percentile, ngưỡng xác suất, NMS, tiling và xuất nhãn hoặc ROI [2]. Thí nghiệm dùng Label Image TIFF cùng kích thước ảnh gốc: nền có ID 0, mỗi instance có ID nguyên dương.

3.1.1 Thông số có ý nghĩa thực nghiệm

Tham số	Giá trị lần chạy tham chiếu	Ý nghĩa/điểm cần kiểm tra
Model	Versatile (fluorescent nuclei)	Phải phù hợp modality; không dùng model H&E cho ảnh huỳnh quang.
Normalize	percentile 1 / 99.8	Cùng percentile đặt trước, nhưng khác thuật toán percentile và xử lý ngưỡng dưới.
Probability threshold	0.479071	Lọc candidate trước NMS; không tối ưu lại trên ba ảnh kiểm chứng.
NMS overlap	0.3	Khử proposal polygon trùng; khác với IoU threshold khi đánh giá.
Tiles / exclude boundary	1 / 2 px	Tile ảnh hưởng bộ nhớ và seam; boundary exclusion thay đổi object nằm sát biên.
Output	Label Image TIFF	Phải cùng kích thước với input/ground truth; không lưu PNG RGB để đo.

3.2 Thực thi và tái lập

Ngoài protocol GUI, nhóm chạy command StarDist đã công bố `de.csbdresden:StarDist_:0.3.0-scijava` trong ImageJ 2.9.0 headless, với ImageJ 1.53t legacy, Eclipse Temurin 8.0.504 và TensorFlow 1.12 CPU. Java 8 tương thích với runtime TensorFlow ImageJ 1.12. Ba ảnh low/median/high được chọn từ test split theo AP của lần chạy Python gốc và giữ cố định; cấu hình dùng các giá trị trong bảng trên. Mỗi lần chạy và runtime được ghi trong `fiji/run_log_dsb2018_reference.csv`; runner Java là `fiji/scripts/FijiStarDistRunner.java`.

Quy trình GUI và macro nằm trong `fiji/workflows/stardist_fiji_protocol.md` và `fiji/macros/stardist_batch.ijm`. Macro cần lệnh từ Recorder tương ứng với phiên bản plugin.

Môi trường Maven/Java 8 trên macOS Intel tái tạo được cả ba nhãn Fiji, trùng từng pixel với nhãn lịch sử. Cấu hình dựng nằm trong `fiji/pom.xml`; log, manifest JAR/SHA-256 và đối chiếu nhãn lưu tại `results/fiji_rerun/`. Runtime tái chạy được giữ riêng với lần chạy lịch sử.

3.2.1 Quy trình GUI

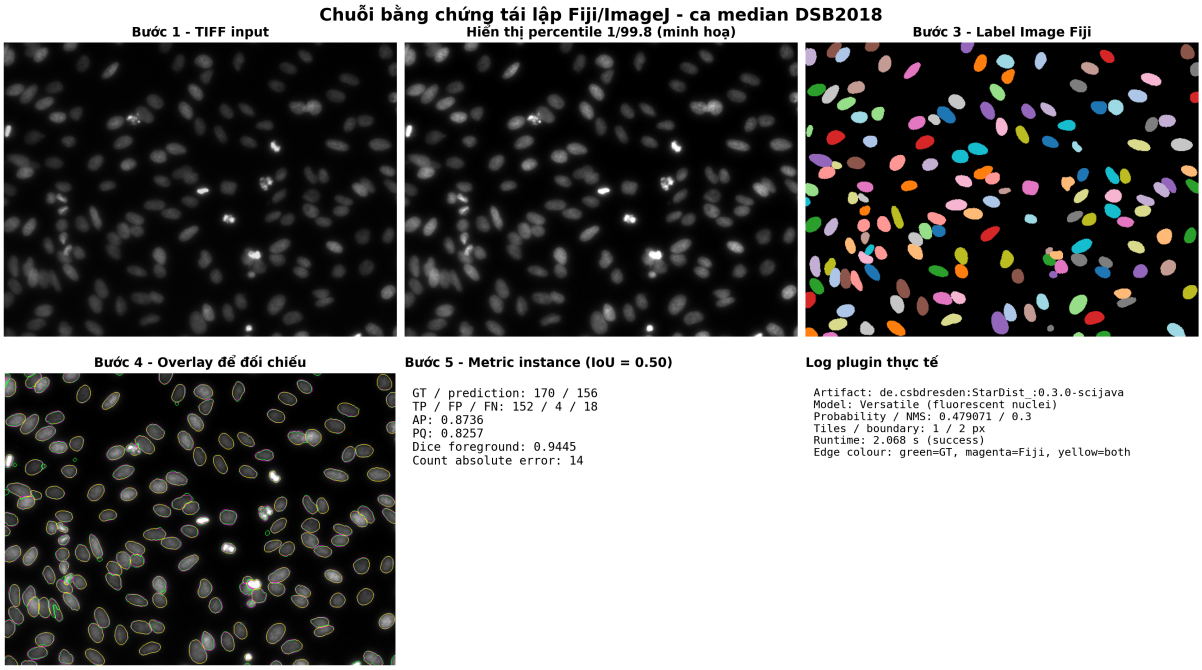
1. Mở TIFF 2D, kiểm tra kích thước, bit-depth và số kênh.
2. Chọn `Plugins > StarDist > StarDist 2D`, nhập cấu hình và bật `Label Image`.
3. Chạy suy luận, kiểm tra nhãn cùng kích thước ảnh và quan sát lỗi tách/gộp.
4. Lưu TIFF nhãn, cấu hình và runtime; chạy evaluator Python trên nhãn đó.

Hình 3.1 tổng hợp đầu vào, nhãn và log của ca median. Ảnh tăng tương phản chỉ phục vụ hiển thị, không phải tensor chuẩn hóa của plugin.

3.2.2 Đo định lượng trong Fiji và giới hạn hiện tại

Fiji có thể đo diện tích foreground từ nhãn TIFF: chuyển bản sao ground truth và prediction thành mask nhị phân, dùng phép AND để tạo giao; trên từng mask chọn `Analyze > Set Measurements > Area`, rồi đo diện tích bằng `Analyze > Measure`. Tính Dice foreground từ diện tích giao và hai foreground theo công thức ở Mục 5.2. Cần cùng kích thước, không nội suy nhãn khi thay đổi hiển thị, và ghi rõ đơn vị pixel hoặc đơn vị đã hiệu chuẩn. Đếm nhãn cần loại nền và đếm ID dương duy nhất; không mặc định mỗi vùng liên thông là một instance nếu nhãn bị phân mảnh.

Nhóm đã đo foreground Dice độc lập bằng API ImageJ 1.53t ở chế độ headless. Chương trình `fiji/scripts/MeasureDice.java` đọc TIFF nhãn, tạo mask nhị phân và đếm pixel foreground, pixel giao bằng histogram của ImageJ. Bảng sau đối chiếu với evaluator Python trên cùng ground truth và nhãn Fiji:



Hình 3.1: Minh chứng từng bước của workflow Fiji/ImageJ trên ca median. Từ trái sang phải: input TIFF, ảnh hiển thị percentile 1/99.8 (minh hoạ), Label Image TIFF StarDist, overlay prediction/ground truth và tham số/runtime từ log.

Ca	Dice ImageJ	Dice Python
low	0.937569	0.937569
median	0.944456	0.944456
high	0.969284	0.969284

Hai cách đo trùng đến ít nhất 10 chữ số thập phân trên ba ca; số pixel và kết quả đầy đủ lưu tại `results/fiji_reference/imagej_dice.csv`. Đây là đo bằng API ImageJ headless, không phải thao tác thủ công trong giao diện Fiji. AP, PQ và matching instance trong bài vẫn do evaluator Python tính; Dice foreground không thay thế đánh giá instance.

3.3 Kiểm chứng Fiji–Python

Metric Fiji so với ground truth được lưu tại `results/fiji_reference/`; đối chiếu trực tiếp Fiji–Python tại `results/cross_platform/`. Kết quả được phân tích ở Mục 6.3.

Chương 4

Cài đặt Python

4.1 Kiến trúc phần mềm

Package `src/xla_gr03` tách tiền xử lý, I/O TIFF, Otsu/Watershed, geometry, StarDist wrapper, metrics và CLI. Mọi output là label integer 2D. Lệnh ‘xla-gr03 segment‘ chạy một phương pháp; ‘xla-gr03 evaluate‘ xuất JSON metric. Unit tests bao phủ geometry, metric, preprocessing, I/O và baseline.

4.2 Phần nhóm tự cài đặt và phần tái sử dụng

Nhóm tự cài đặt EDT chuẩn hoá theo instance, ray distance, polygon decode, polygon NMS raster hoá, IoU matrix và hàm đánh giá instance dùng bộ giải Hungarian của SciPy. Các hàm geometry là bản tham chiếu giáo dục, chưa thay hậu xử lý chính thức trong inference. CNN pretrained và hậu xử lý suy luận dùng thư viện StarDist chính thức [7].

4.3 Cài đặt các bước hình học

Các hàm trong `src/xla_gr03/geometry.py` minh họa các thành phần hình học của StarDist. Đầu vào là ảnh nhãn nguyên 2D, trong đó 0 là nền và mỗi số nguyên dương xác định một nhân. Hàm kiểm tra số chiều, kiểu số nguyên và giá trị không âm trước khi xử lý.

4.3.1 Bản đồ khoảng cách chuẩn hóa

Với mỗi nhãn m, tạo mask $M_m(q) = 1$ khi $Y(q) = m$ và bằng 0 ở các vị trí khác. Biến đổi khoảng cách Euclid cho biết khoảng cách từ pixel trong mask đến pixel nền gần nhất. Giá trị đích tại mỗi instance là

$$p_m(q) = \frac{D_m(q)}{\max_{u \in M_m} D_m(u)}, \qquad q \in M_m. \tag{4.1}$$

Nền nhận giá trị 0. Chuẩn hóa theo từng nhân giúp các nhân lớn và nhỏ cùng có cực đại bằng 1. Khi instance chạm biên ảnh, triển khai không tự coi toàn bộ vùng ngoài ảnh là nền; trường hợp một instance phủ kín ảnh được xử lý riêng.

4.3.2 Tia, đa giác và NMS tham chiếu

Với mỗi pixel foreground, hàm `star_distances` duyệt các hướng cách đều, tăng khoảng cách theo bước lấy mẫu và dừng khi vượt ảnh hoặc gặp nhãn khác. Mỗi dãy khoảng cách được giải mã thành các đỉnh theo công thức (2.5), sau đó raster hóa để kiểm tra diện tích và chồng lấn. Cách duyệt từng pixel, từng tia ưu tiên minh bạch, phù hợp ảnh nhỏ và kiểm thử; nó không thay thế hậu xử lý tối ưu của thư viện trong benchmark. Việc làm tròn tọa độ và bước lấy mẫu có thể làm sai khác biên so với triển khai tối ưu.

NMS tham chiếu sắp proposal theo điểm giảm dần, chọn proposal chưa bị loại và so sánh với các proposal còn lại bằng tỉ lệ giao trên diện tích nhỏ hơn. Nếu vượt ngưỡng, proposal điểm thấp hơn bị loại. Thứ tự này cần được giữ cố định để kết quả có thể kiểm tra lại. Các unit test cho geometry, ghép cặp và nhãn nằm trong `tests/`; CNN và weight trong benchmark lấy từ StarDist chính thức.

4.3.3 Baseline và metric do nhóm lập trình

Otsu tạo mask foreground bằng ngưỡng toàn cục, lọc vùng nhỏ rồi gán nhãn các thành phần liên thông. Watershed tính EDT của mask, tìm marker bằng cực đại địa phương và phân vùng trên $-D$ trong phạm vi foreground. Các hàm xuất TIFF nhãn nguyên và dùng chung evaluator. Bộ đánh giá tạo ma trận IoU, ghép một–một bằng Hungarian, sau đó tính TP, FP, FN, AP, F1 và PQ tại từng ngưỡng; Dice được đo trên hợp foreground. Pipeline và logic matching nằm trong `src/xla_gr03/` và `tests/`.

4.3.4 Ví dụ kiểm tra thủ công

Giả sử ground truth có một nhân diện tích 100 pixel và prediction tách đúng vùng đó thành hai mảnh không giao nhau, mỗi mảnh 50 pixel. Hai IoU bằng 0.5. Tại ngưỡng $\tau = 0.5$, ghép một–một chỉ chọn được một cặp: $TP = 1$, $FP = 1$, $FN = 0$. Vì vậy $F1 = 2/3$ và $AP_{0.5} = 1/2$. Hợp foreground của hai mảnh vẫn trùng ground truth nên Dice foreground bằng 1. Ví dụ cho thấy Dice cao không phát hiện được lỗi tách instance; các test split/merge kiểm tra riêng tính chất này trong bộ metric.

Để kiểm tra NMS, xét proposal A có diện tích 100 và proposal B nằm hoàn toàn trong A với diện tích 25 pixel. Overlap StarDist bằng $25/25 = 1$, còn IoU bằng $25/100 = 0.25$. Nếu ngưỡng NMS là 0.3 và A có điểm cao hơn, B bị loại theo overlap. Nếu nhầm IoU đánh giá với overlap NMS, B sẽ bị giữ lại. Khi triển khai, cần cố định thứ tự xử lý các proposal và đơn vị tọa độ trước khi so sánh các phiên bản.

4.4 Môi trường

Thực nghiệm dùng Python 3.11.9, TensorFlow 2.15.1 và StarDist 0.9.1 trên CPU Intel/macOS 15.7. Các dependency được ghi trong `requirements-model.txt`; phiên bản thực tế và hash checkpoint được lưu cùng kết quả. Lệnh cài đặt và kiểm thử nằm ở Phụ lục B.

Chương 5

Thiết kế thí nghiệm và metric

5.1 Dữ liệu và cấu hình

Bộ DSB2018 do StarDist phân phối gồm 497 cặp ảnh–nhãn TIFF: 447 ảnh train và 50 ảnh test [1]. Nhãn ground truth chứa ID instance. Dữ liệu lưu tại `data/dsb2018/dsb2018/`, kèm manifest SHA-256.

Từ train split, chọn 30 ảnh validation với seed 42 để khảo sát ngưỡng và chọn `min_distance` của Watershed trong $\{3, 5, 7, 9, 11\}$. Giá trị 9 đạt AP@0.50 validation cao nhất. Danh sách ảnh nằm trong `results/dsb2018_benchmark/validation_images.csv`; 50 ảnh test không tham gia chọn tham số.

StarDist dùng checkpoint `2D_versatile_fluo`, percentile $1/99.8$, ngưỡng xác suất 0.479071 và NMS 0.3 mặc định của model.

5.2 Matching và metric

Gọi G_i là instance ground truth thứ i và P_j là instance prediction thứ j. Trước hết code bỏ nhãn nền 0, lập ma trận giao nhau pixel-wise rồi tính

$$J_{ij} = \mathrm{IoU}(G_i, P_j) = \frac{|G_i \cap P_j|}{|G_i \cup P_j|}. \tag{5.1}$$

Hungarian assignment ưu tiên tối đa số cặp một–một có $J_{ij} \geq \tau$, sau đó tối đa tổng IoU. Cặp đạt ngưỡng là TP; ground truth chưa ghép là FN, prediction chưa ghép là FP. Một prediction gộp hai nhân chỉ khớp tối đa một nhân; nhân còn lại là FN.

Tại mỗi $\tau \in \{0.50, 0.75, 0.90\}$, các chỉ số là

$$P = TP/(TP + FP), \qquad R = TP/(TP + FN), \tag{5.2}$$

$$F1 = 2TP/(2TP + FP + FN), \qquad AP_\tau = TP/(TP + FP + FN). \tag{5.3}$$

Ở đây AP_τ là tên metric theo benchmark StarDist/Data Science Bowl, không phải area under precision–recall curve. $\tau = 0.50$ trả lời "có nhận đúng object không", còn $\tau = 0.90$ đòi biên khớp rất chính xác.

Panoptic Quality kết hợp nhận diện và chất lượng mask [3]:

$$PQ_\tau = \frac{\sum_{(i,j)\in \mathrm{TP}} J_{ij}}{TP + \frac{1}{2}FP + \frac{1}{2}FN}. \tag{5.4}$$

Ngoài ra, foreground Dice $2|F_G \cap F_P|/(|F_G| + |F_P|)$ và foreground IoU đo vùng nhân hợp nhất, nhưng không phân biệt ID instance. Sai số đếm là $|N_P - N_G|$. Các metric tính riêng từng ảnh rồi tổng hợp trung bình $\pm$ độ lệch chuẩn (SD).

5.3 Giao thức đánh giá

Otsu, Watershed và StarDist Python được đánh giá trên cùng 50 ảnh test. Fiji chạy riêng trên ba ca low/median/high cố định để đối chiếu triển khai. Runtime Python gồm chuẩn hóa, suy luận và hậu xử lý sau warm-up, không gồm I/O hay tải model. Runtime Fiji gồm khởi tạo process Java và model, nên không dùng để so tốc độ trực tiếp với Python.

Script `scripts/run_dsb2018_benchmark.py` sinh kết quả 50 ảnh; `scripts/evaluate_fiji_reference.py` đánh giá nhãn Fiji. Cấu hình, thiết bị, hash dữ liệu và checkpoint được lưu trong provenance và manifest.

Chương 6

Kết quả và thảo luận

6.1 So sánh trên 50 ảnh test

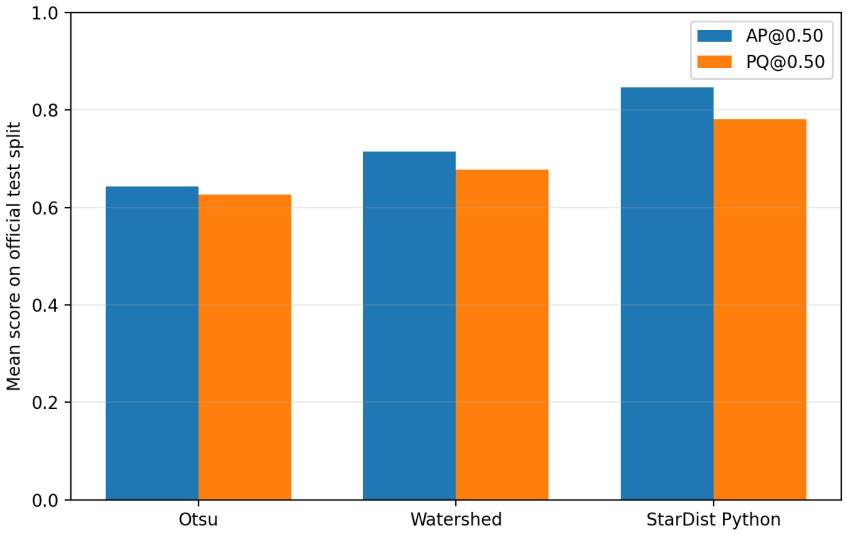
Bảng 6.1 trình bày chất lượng instance; Bảng 6.2 bổ sung Dice foreground, sai số đếm và runtime.

Bảng 6.1: Metric instance trên cùng 50 test images. Fiji được đánh giá riêng ở bảng đối chiếu.

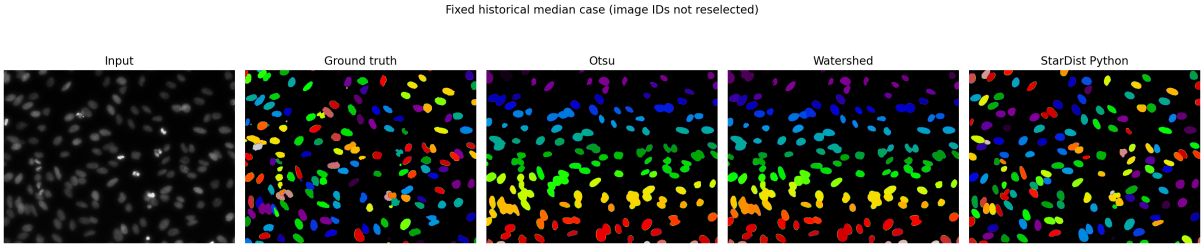
Method	AP@.50	AP@.75	AP@.90	PQ@.50
Otsu	0.643 ± 0.238	0.453 ± 0.276	0.223 ± 0.221	0.627 ± 0.208
Watershed	0.715 ± 0.199	0.501 ± 0.266	0.214 ± 0.207	0.677 ± 0.166
StarDist Python	0.846 ± 0.132	0.692 ± 0.238	0.305 ± 0.313	0.782 ± 0.117

Bảng 6.2: Metric foreground, sai số đếm và runtime Python/baseline, mean $\pm$ SD trên 50 ảnh.

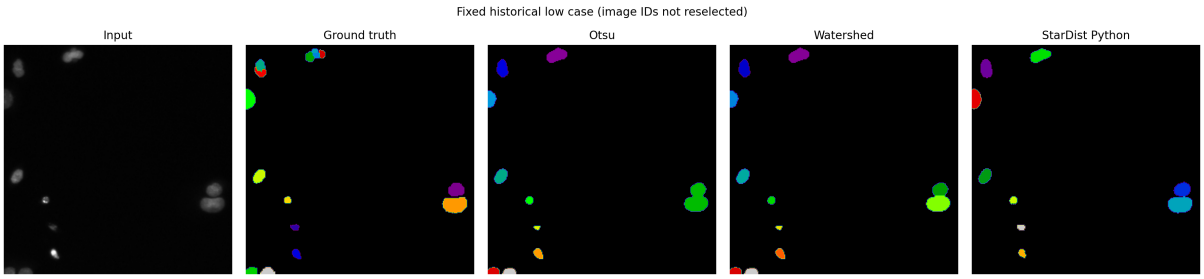
Method	Dice foreground	MAE count	Runtime (s/image)
Otsu	0.895 ± 0.070	14.06 ± 25.28	0.0327 ± 0.0232
Watershed	0.894 ± 0.071	6.94 ± 10.56	0.0647 ± 0.0462
StarDist Python	0.915 ± 0.050	3.48 ± 3.79	0.3277 ± 0.2098



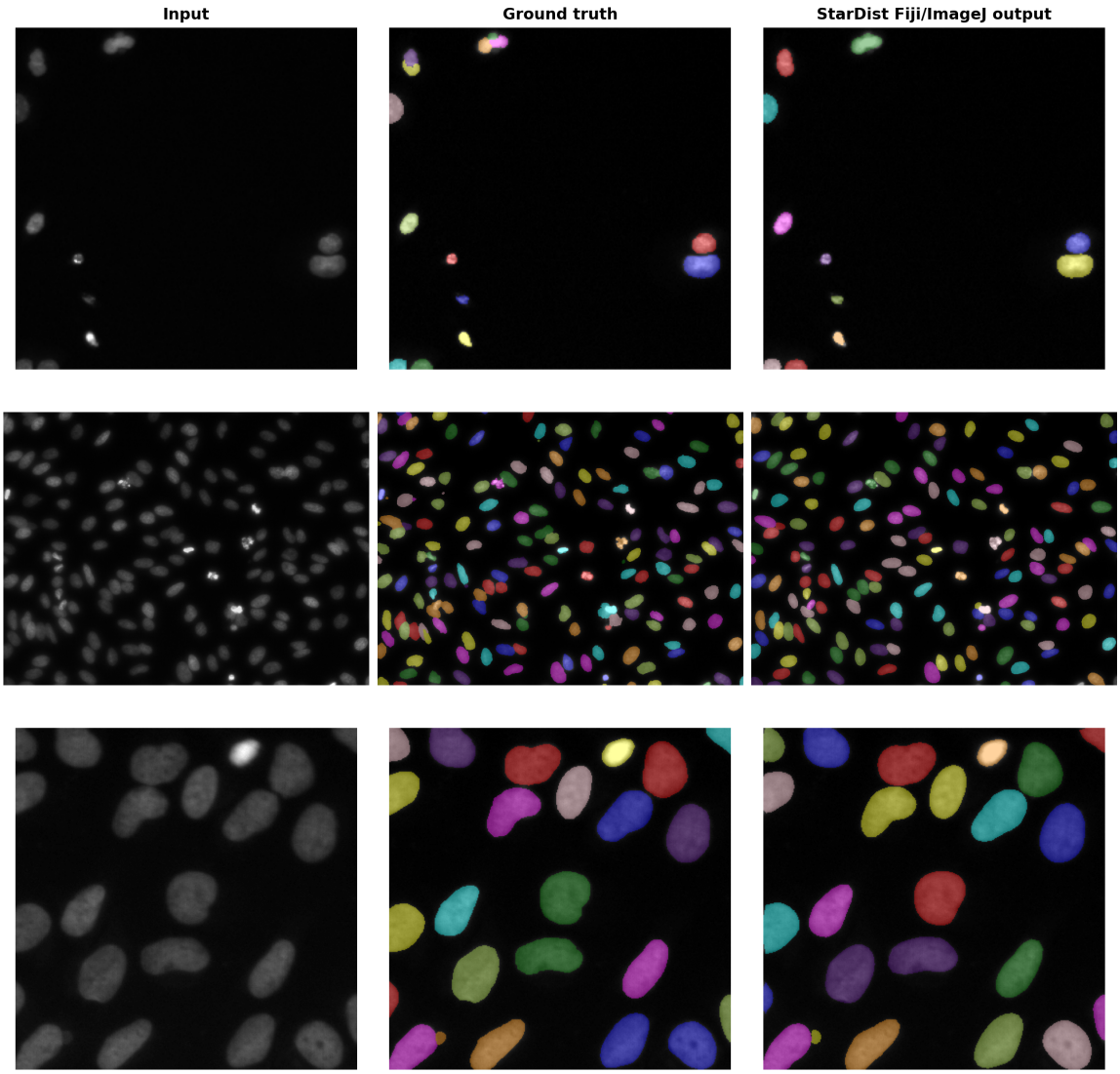
Hình 6.1: Mean AP@0.50 và PQ@0.50 trên 50 ảnh test DSB2018. Độ lệch chuẩn được trình bày trong Bảng 6.1.



Hình 6.2: Ca median được giữ cố định từ lần chạy gốc có clipping, không chọn lại theo kết quả mới. Màu biểu diễn ID instance, không phải lớp semantic. Otsu gộp nhiều nhân tiếp xúc; Watershed tách được một phần; StarDist bám theo instance ground truth sát hơn.



Hình 6.3: Ca low cố định từ lần chạy gốc; tên ca không biểu thị thứ hạng AP sau cập nhật.



Hình 6.4: Ba output Label Image thực tế của plugin StarDist Fiji/ImageJ: ca low, median và high đã chọn trước, đặt cạnh input và ground truth. Mỗi màu là một ID instance.

6.2 Phân tích kết quả

Trên 50 ảnh test, StarDist đạt AP@0.50 0.846 ± 0.132 và PQ 0.782 ± 0.117, so với Watershed (0.715 ± 0.199; 0.677 ± 0.166) và Otsu (0.643 ± 0.238; 0.627 ± 0.208). Hai baseline vẫn phủ foreground khá tốt nhưng AP instance thấp hơn: kết quả này cho thấy Dice toàn foreground không đủ để đánh giá tách object. Watershed giảm MAE đếm từ 14.06 xuống 6.94 so với Otsu; StarDist đạt 3.48. Ở $\tau = 0.90$, metric của cả ba giảm mạnh, phản ánh yêu cầu chính xác biên rất nghiêm ngặt. StarDist có runtime CPU 0.3277 s/ảnh so với 0.0647 s/ảnh của Watershed, đổi lại tách instance tốt hơn trên benchmark này. Trên ba ca kiểm chứng, plugin Fiji/ImageJ đạt AP@0.50 0.801 ± 0.244, PQ 0.788 ± 0.160 và Dice 0.950 ± 0.017; kết quả này được tổng hợp riêng, không dùng để xếp hạng với mẫu 50 ảnh.

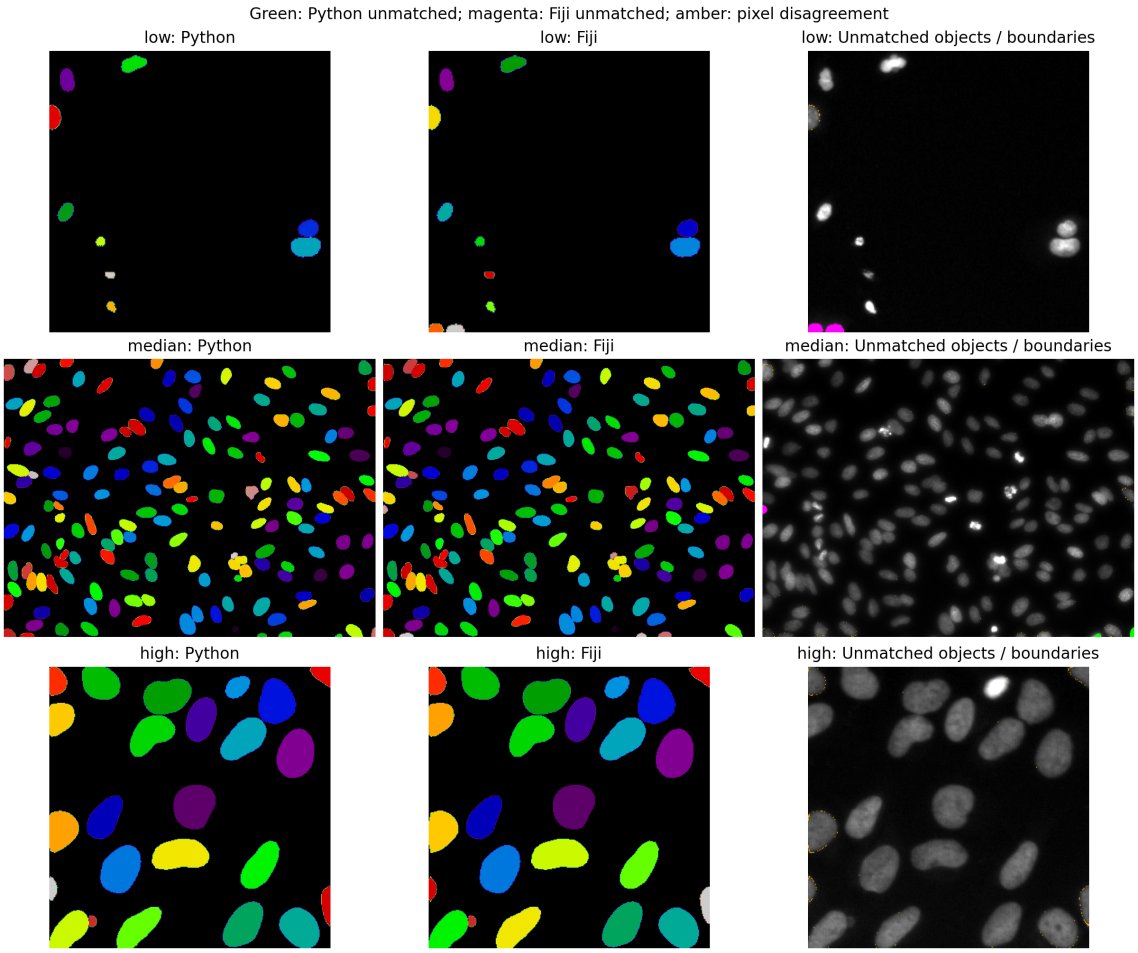
6.3 Đối chiếu trực tiếp Fiji–Python

Script `scripts/compare_fiji_python.py` ghép trực tiếp label Python với label Fiji trên ba image ID cố định. Nhãn Python chỉ là tham chiếu so sánh, không thay ground truth sinh học. Tiêu chí mô tả dùng trong script là F1 tại IoU 0.50 ít nhất 0.95 và Dice foreground ít nhất 0.95. Đây là ngưỡng kỹ thuật cho ba ca, không phải kiểm định tương đương thống kê.

Ca	N_{Python}	N_{Fiji}	F1@.50	IoU ghép TB	Dice
low	9	11	0.9000	0.9956	0.9405
median	157	156	0.9904	0.9975	0.9970
high	24	24	1.0000	0.9903	0.9980

Ca median và high đạt tiêu chí mô tả; ca low không đạt: Python có 9 object, Fiji có 11 object, F1 0.900 và Dice 0.940. Ca low còn hai object Fiji không ghép được với Python. Overlay đánh dấu object Python chưa ghép bằng xanh lá, Fiji chưa ghép bằng tím, sai khác pixel/biên sau matching bằng vàng.

Python dùng percentile tuyến tính NumPy và phép affine không clipping. CSBDeep Java 0.6.0 dùng histogram percentile và chặn giá trị âm về 0 dù `clip=false`. Fiji còn đặt exclude boundary 2 pixel; hai nền tảng dùng TensorFlow khác phiên bản. Những khác biệt có thật này được lưu trong CSV và provenance; thí nghiệm hiện tại chưa cô lập từng nguyên nhân nên không quy hai object chênh lệch cho riêng một bước.

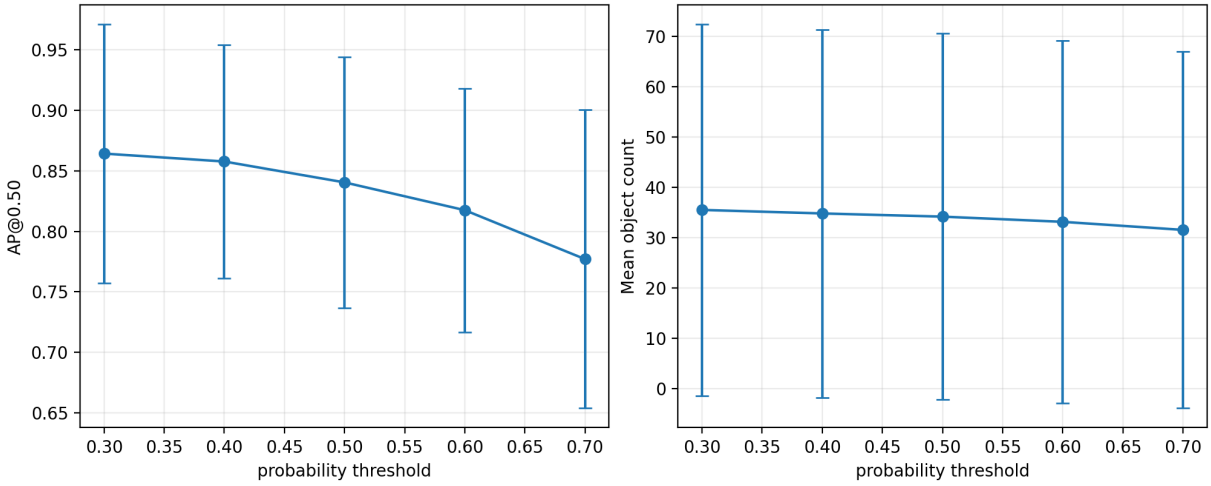


Hình 6.5: Sai khác trực tiếp giữa hai đầu ra. ID nhãn được ghép một-một trước khi đo pixel agreement trên hợp foreground, tránh kết luận sai do đổi thứ tự ID.

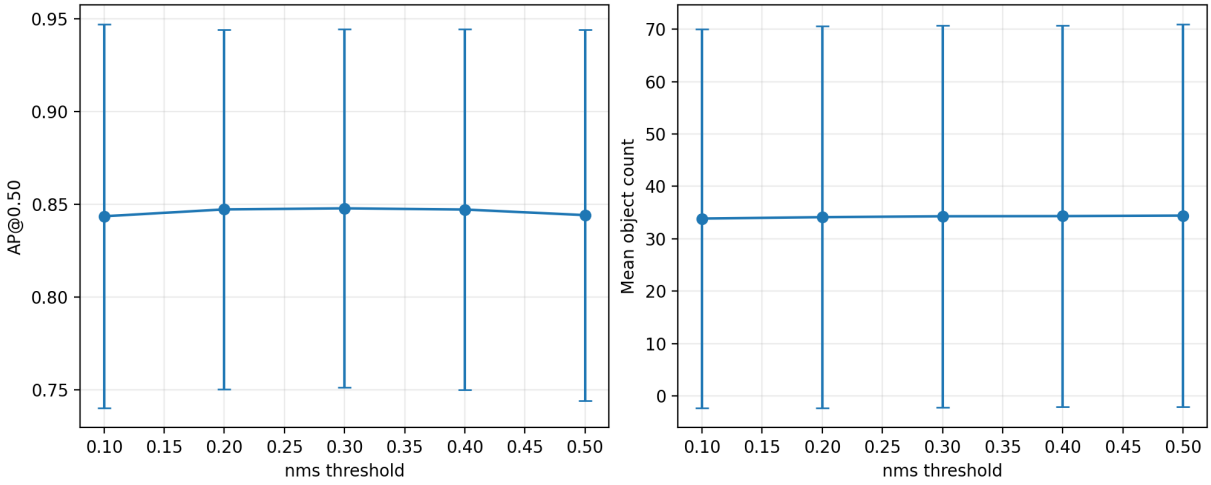
6.4 Ảnh hưởng ngưỡng xác suất và NMS

Script `scripts/run_validation_ablation.py` đánh giá 5 probability threshold và 5 NMS threshold trên đúng 30 ảnh train-validation đã chốt. Mỗi ảnh chỉ tính CNN một lần rồi thay tham số hậu xử lý; do đó sweep không báo thời gian inference. CSV lưu trung bình và SD của AP, F1, PQ, số object, FP, FN và sai số đếm.

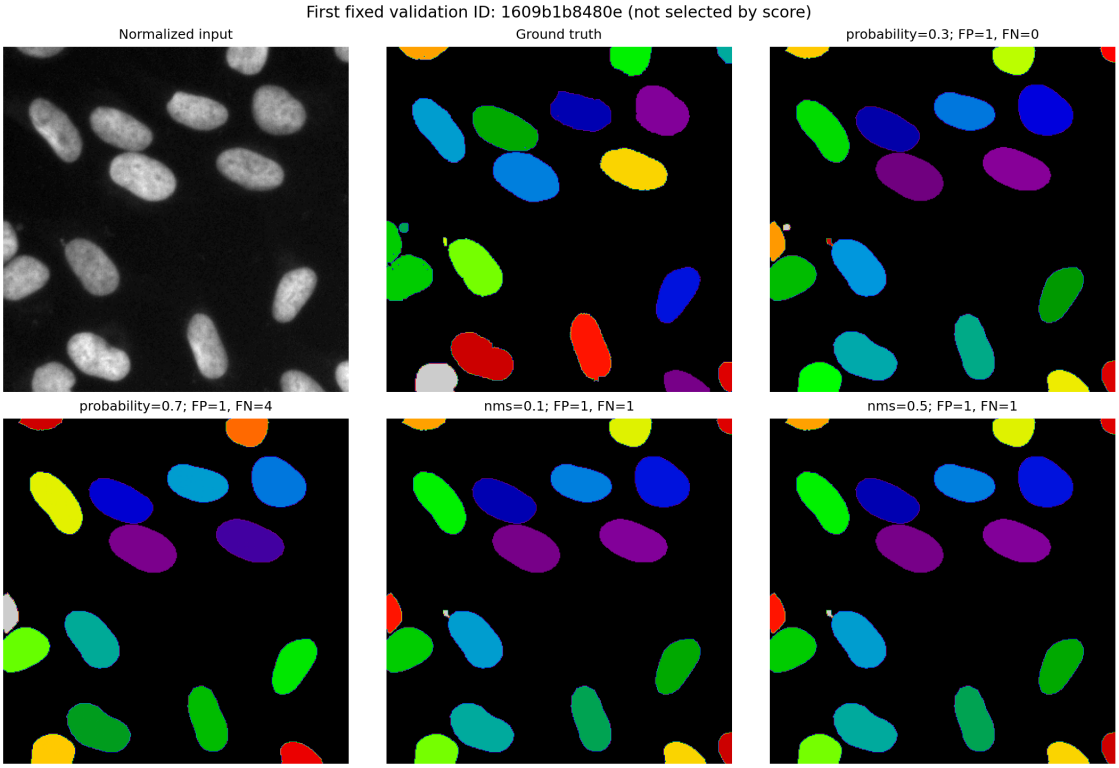
Khi tăng probability threshold từ 0.3 lên 0.7, AP@0.50 validation giảm từ 0.864 xuống 0.777; số object trung bình giảm từ 35.47 xuống 31.50; FN tăng từ 3.13 lên 6.47. Ngưỡng cao bỏ thêm candidate yếu và có thể bỏ sót nhân. Quét NMS giữ probability mặc định 0.479071; quét probability giữ NMS 0.3. Benchmark test vẫn giữ ngưỡng model công bố, không đổi theo sweep này.



Hình 6.6: Probability sweep trên 30 ảnh validation; error bar là SD giữa ảnh.



Hình 6.7: NMS sweep trên cùng validation, giữ probability mặc định.



Hình 6.8: Ví dụ ở ID validation đầu tiên đã chốt, không chọn theo score. Hai đầu mút probability/NMS kèm FP/FN minh hoạ candidate được giữ/bỏ khi đổi ngưỡng. Màu là ID instance; kết luận định lượng vẫn dựa toàn bộ 30 ảnh.

Chương 7

Kết luận

Trên 50 ảnh test DSB2018, StarDist đạt AP@0.50 0.846 ± 0.132, PQ 0.782 ± 0.117 và MAE đếm 3.48, tốt hơn Otsu và Watershed trong cấu hình đã đánh giá. Đối chiếu Fiji–Python đạt tiêu chí ở hai trong ba ca; ca low còn chênh lệch hai instance. Ngưỡng xác suất cao làm tăng bỏ sót trên validation, trong khi ảnh hưởng của NMS nhỏ hơn trong khoảng đã quét.

7.1 Giới hạn

- Checkpoint pretrained có nguồn huấn luyện từ DSB2018, nhưng chưa xác minh mức trùng lặp ID với 50 ảnh test. Kết quả chưa chứng minh khả năng tổng quát hóa trên dữ liệu độc lập; chưa thực hiện fine-tuning.
- Đối chiếu Fiji–Python chỉ gồm ba ca. Chuẩn hóa, xử lý biên và phiên bản TensorFlow khác nhau; thí nghiệm chưa cô lập tác động từng yếu tố.
- Runtime hai nền tảng có phạm vi đo khác nhau. Thứ hạng tốc độ chỉ áp dụng cho ba phương pháp Python trên cùng cấu hình CPU.
- Giả thiết star-convex hạn chế biểu diễn nhân dạng vòng, phân nhánh hoặc lõm sâu; thay đổi miền ảnh cũng có thể làm giảm chất lượng.

7.2 Hướng phát triển

Hướng phát triển ưu tiên là đánh giá trên dataset ngoài DSB2018 và đồng nhất chuẩn hóa, xử lý biên giữa hai nền tảng trước khi mở rộng kiểm chứng.

Tài liệu tham khảo

[1] Juan C. Caicedo, Allen Goodman, Kyle W. Karhohs, et al. Nucleus segmentation across imaging experiments: the 2018 data science bowl. *Nature Methods*, 16(12):1247–1253, 2019. doi: 10.1038/s41592-019-0612-7.

[2] ImageJ. Stardist. https://imagej.net/plugins/stardist, 2026. Accessed: 2026-10-03.

[3] Alexander Kirillov, Kaiming He, Ross Girshick, Carsten Rother, and Piotr Dollár. Panoptic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9404–9413, 2019. doi: 10.1109/CVPR.2019.00963.

[4] Nobuyuki Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979. doi: 10.1109/TSMC.1979.4310076.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, pages 234–241. Springer, 2015. doi: 10.1007/978-3-319-24574-4_28.

[6] Uwe Schmidt, Martin Weigert, Coleman Broaddus, and Gene Myers. Cell detection with star-convex polygons. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*, pages 265–273. Springer, 2018. doi: 10.1007/978-3-030-00934-2_30.

[7] StarDist contributors. Stardist – object detection with star-convex shapes. https://github.com/stardist/stardist, 2026. Accessed: 2026-10-03.

[8] Luc Vincent and Pierre Soille. Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 13(6):583–598, 1991. doi: 10.1109/34.87344.

Phụ lục A

Phân công công việc

Cả ba thành viên cùng thực hiện phần lý thuyết, mô hình toán và tài liệu.

Thành viên	Công việc và artefact
Nguyễn Hồng Quang B23DCVT361	Package Python, test, CLI và `scripts/run_experiment.py`.
Nguyễn Trọng Nam Khánh B23DCCE052	Workflow Fiji/ImageJ: plugin StarDist, screenshot thao tác, macro, run log và label TIFF.
Vũ Dũng B23DCVT110	Tìm kiếm, thu thập và chuẩn bị dataset; tổ chức split và ground truth.

Phụ lục B

Tái lập thực nghiệm

B.1 Cài môi trường và kiểm thử

```
python -m venv .venv
source .venv/bin/activate  # macOS/Linux
python -m pip install -r requirements-model.txt
python -m pip install -e ".[dev]"
python -m pytest -q
python -m pytest -m integration -q
python -m ruff check src tests scripts
```

B.2 Tạo lại kết quả và artefact

```
python scripts/run_dsb2018_benchmark.py
python scripts/run_validation_ablation.py
python scripts/evaluate_fiji_reference.py
python scripts/compare_fiji_python.py
python scripts/build_fiji_workflow_evidence.py
python scripts/build_report_data.py
```

Dữ liệu TIFF đã có trong repository; script tải model qua API chính thức nếu chưa có cache. Để tái chạy Fiji, xem `docs/06_tai_lap.md` và `scripts/run_fiji_reference.py`.

B.3 Mã nguồn tham chiếu

- Matching và metric: `src/xla_gr03/metrics.py`.
- Hình học star-convex: `src/xla_gr03/geometry.py`.
- Benchmark và validation: `scripts/run_dsb2018_benchmark.py`, `scripts/run_validation_ablation.py`.
- Đối chiếu nền tảng: `scripts/compare_fiji_python.py`.
- Runner Fiji: `fiji/scripts/FijiStarDistRunner.java`.